Pashov Audit Group

Opinion Security Review

March 24th 2026 - March 26th 2026

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Opinion

Opinion is a prediction market protocol that manages the resolution and dispute of on-chain market outcomes. The DisputeResolver contract handles the full lifecycle of dispute resolution, allowing participants to challenge market outcomes within a 24-hour window, triggering a committee re-vote, and distributing stakes and rewards based on the final outcome.

5. Executive Summary

A time-boxed security review of the **OpinionLabs/centauri** repository was done by Pashov Audit Group, during which **JCN, Valves Security, VictoryGod, t.aksoy** engaged to review **Opinion**. A total of **12** issues were uncovered.

Protocol Summary	
Project Name	Opinion
Protocol Type	Prediction market
Timeline	March 24th 2026 - March 26th 2026

Review commit hash:
- cd92a6c80499f524fcd20cc8a9dd70c171b47451 (OpinionLabs/centauri)

Fixes review commit hash:
- 872af1059b8eb24cdfc256a183d579adb54c5b94 (OpinionLabs/centauri)

Scope

`DisputeResolver.sol`

6. Findings

Findings count

Severity	Amount
Low	12
Total findings	**12**

Summary of findings

ID	Title	Severity	Status
[L-01]	Pausing blocks reward recovery for upheld dispute	Low	**Resolved**
[L-02]	Deadline manipulation still possible via `globalReVoteWindow` update	Low	Acknowledged
[L-03]	Too early limbo no expiry if committee never re-submits	Low	Acknowledged
[L-04]	Resolution multiSig change orphans active revotes	Low	Acknowledged
[L-05]	WrongResult and TooEarly are mixed incorrectly	Low	Acknowledged
[L-06]	Dispute timeouts always favor the disputor	Low	Acknowledged
[L-07]	Blocklisting token permanently locks claim funds	Low	Acknowledged
[L-08]	Dispute slot front-running via single-disputor model	Low	Acknowledged
[L-09]	OracleProxy preregistration not enforced at initialize	Low	Acknowledged
[L-10]	Reward_manager non-deposit creates permanent dispute censorship	Low	**Resolved**
[L-11]	OracleProxy.resolve() revert permanently blocks finalization	Low	Acknowledged
[L-12]	Operator pause-finalize combo eliminates dispute window	Low	Acknowledged

Low findings

[L-01] Pausing blocks reward recovery for upheld dispute

Description

The `withdrawRewardPool` function does not include a `whenNotPaused` modifier since this function should allow the admin to recover rewards even when the contract is paused. However, there is a check that requires `claim` to be invoked first for upheld disputes:

```
    function withdrawRewardPool(bytes32 questionId) external onlyRole(REWARD_MANAGER_ROLE)
nonReentrant {
        Resolution storage res = _resolutions[questionId];

        // Validate resolution exists
        if (res.receivedAt == 0) revert ResolutionNotFound(questionId);

        // If a disputor exists, check claim state
        if (res.disputor != address(0)) {
            // Cannot withdraw if dispute succeeded (reward belongs to disputor)
            if (res.disputeSucceeded) {
                revert RewardNotReclaimable(questionId);
            }

            // For upheld disputes, require claim processed first
            if (!res.claimed) {
                revert RewardNotReclaimable(questionId);
            }
        }
```

The `claim` function has a `whenNotPaused` modifier, which means that rewards cannot be withdrawn for upheld disputes (via `withdrawRewardPool`) when the contract is paused. For upheld disputes, claiming the stake amount (via `claim`) and reward amount (via `withdrawRewardPool`) are independent actions that modify different states. The current implementation unnecessarily blocks the admin from withdrawing rewards during a contract pause.

Consider updating `withdrawRewardPool` to allow the rewards to be recovered for upheld disputes directly.

[L-02] Deadline manipulation still possible via `globalReVoteWindow` update

Description

When a user disputes a resolution, the `reVoteDeadline` is computed via a configured question-specific time or via a global `globalReVoteWindow` time:

```
        QuestionDeadlineConfig storage cfg = _questionDeadlines[questionId];
        res.reVoteDeadline = block.timestamp + (cfg.configured ? cfg.reVoteWindow :
globalReVoteWindow);
```

When an operator attempts to update a question-specific deadline, the function will revert if a resolution for said question has already been logged to prevent post-resolution deadline manipulation:

```
    function configureQuestionDeadline(
        bytes32 questionId,
        uint256 _disputeWindow,
        uint256 _reVoteWindow
    ) external onlyRole(OPERATOR_ROLE) nonReentrant whenNotPaused {
        if (_disputeWindow < MIN_WINDOW_DURATION || _disputeWindow > MAX_WINDOW_DURATION)
            revert InvalidWindowDuration(_disputeWindow, MIN_WINDOW_DURATION,
MAX_WINDOW_DURATION);
        if (_reVoteWindow < MIN_WINDOW_DURATION || _reVoteWindow > MAX_WINDOW_DURATION)
            revert InvalidWindowDuration(_reVoteWindow, MIN_WINDOW_DURATION,
MAX_WINDOW_DURATION);

        // Prevent modifying config after resolution exists (M-01: reVoteWindow
        // is read dynamically at dispute() time, so changing it post-resolution
        // could allow deadline manipulation)
        if (_resolutions[questionId].receivedAt != 0) revert
ResolutionAlreadyExists(questionId);
...
```

However, post-resolution deadline manipulation is still possible via the `globalReVoteWindow`, if a question does not have a configuration enabled.

A question using global defaults gets whichever `globalReVoteWindow` is set at the moment the dispute is filed, not at the moment the resolution was created. If the admin reduces the global window between resolution creation and dispute filing, the committee may have far less time to re-vote than was implied when the resolution was logged.

Consider snapshotting the re-vote deadline window for the resolution at creation time. This deadline window will either be the currently configured deadline for the question or the currently configured global deadline. During `dispute`, the `reVoteDeadline` can then be initialized with this snapshotted deadline window plus the current timestamp.

Alternatively, consider updating documentation to describe this behavior so integrators can be aware that the re-vote deadline can still be manipulated after the resolution has been created.

[L-03] Too early limbo no expiry if committee never re-submits

Description

When a dispute times out via `_timeoutDispute()` (line 830), the status becomes `TooEarly` and `disputeSucceeded = true` (line 849). The disputor can claim their 2× reward. However, the resolution remains in `TooEarly` indefinitely until the committee re-submits via `resolve()`. There is no expiry on the `TooEarly` state.

`DisputeResolver.sol` — `_timeoutDispute()` (line 830), `resolve()` Too Early case (line 266)

If the committee never re-submits (disbanded, lost access, or simply forgets), the question remains in limbo forever.

Beyond stake accounting, while a resolution remains in `TooEarly`, the final payout is never forwarded to `OracleProxy` / ConditionalTokens, leaving the prediction market permanently unresolved and position holders unable to redeem their shares.

Recommendations

Add an `autoExpireTooEarly()` function that, after a configurable timeout, either marks the resolution as Finalized with the original payouts or marks it as permanently unresolved and emits an event for off-chain cleanup.

Acknowledgement Comment

> This is intentional. The exact timing of when a market resolves cannot be predicted, and if a resolution was deemed "too early," the system should wait for the committee to provide a definitive answer rather than auto-expiring into an incorrect state.

[L-04] Resolution multiSig change orphans active revotes

Description

When the `resolutionMultiSig` is changed, `RESOLVER_ROLE` is revoked from the old address and granted to the new one (lines 566–568). Any resolution currently in `Disputed` status awaiting a re-vote from the old multisig becomes orphaned — the old multisig lost `RESOLVER_ROLE` and the new one may not know about the pending question.

`DisputeResolver.sol` — `setResolutionMultiSig()` (line 561)

Recommendations

Before allowing the change, check that no resolutions are in `Disputed` status, or migrate active re-vote responsibilities to the new multisig with explicit notification.

Acknowledgement Comment

> We will incorporate guidance on this behavior into the deployment and operational guide. The admin should ensure no resolutions are in Disputed status before changing the resolutionMultiSig address.

[L-05] WrongResult and TooEarly are mixed incorrectly

Description

When a dispute times out, the contract always sets the result to TooEarly, even if the original dispute was about a wrong result. This mixes two very different meanings: one where the outcome is incorrect, and one where the event has not happened yet. Losing this distinction makes the system harder to reason about and can lead to incorrect handling in later stages.

There is an additional issue when a user disputes specifically for TooEarly. In this case, the user is not claiming that the result is wrong, but only that it is premature. However, when the committee revotes, `_resolveDispute()` simply compares payouts. If the payouts remain the same, the dispute is marked as failed (Upheld), and the user is penalized by losing their stake. This is incorrect behavior because the user did not oppose the final result itself, only its timing. As a result, a valid “TooEarly” dispute can still be punished unfairly.

This shows that the system does not properly separate dispute types. Both WrongResult and TooEarly disputes are processed through the same payout comparison logic, even though they represent fundamentally different claims.

Recommendations

Handle TooEarly disputes differently from WrongResult disputes. For TooEarly, the logic should validate timing rather than comparing payouts, and users should not be penalized if the final result matches but was previously premature.

Acknowledgement Comment

> We acknowledge the distinction between dispute reasons. However, users should not be penalized if the final result matches but was previously premature. The current design keeps claim logic simple and avoids complexity in separating dispute type outcomes, which we consider an acceptable trade-off at this stage.

[L-06] Dispute timeouts always favor the disputor

Severity

Impact: High

Likelihood: Low

Description

When the committee fails to finish the re-vote process for a disputed proposal, the admin is able to settle the resolution via `timeoutDispute`.

This function will set `disputeSucceeded = true` unconditionally. If the resolution has a disputor, this means that the disputor will then be able to claim 2 times their stake via the contract:

```
    function _timeoutDispute(bytes32 questionId) internal {
        Resolution storage res = _resolutions[questionId];

        // Validate resolution exists
        if (res.receivedAt == 0) revert ResolutionNotFound(questionId);

        // Validate status is Disputed
        if (res.status != DisputeStatus.Disputed) {
            revert InvalidStatus(questionId, res.status);
        }

        // Timeout is inclusive of the deadline second: at exactly reVoteDeadline,
        // timeout wins and _resolveDispute() is already closed (H-01 fix).
        if (block.timestamp < res.reVoteDeadline) {
            revert ReVoteWindowOpen(questionId, res.reVoteDeadline);
        }

        // Agent committee didn't reach consensus in time - dispute succeeds
        res.status = DisputeStatus.TooEarly;
        res.disputeSucceeded = true;
```

```
    function claim(bytes32 questionId) external nonReentrant whenNotPaused {
...
        if (res.disputeSucceeded) {
            // Dispute succeeded - only disputor can claim
            if (msg.sender != res.disputor) {
                revert NotDisputor(msg.sender, res.disputor);
            }
            // Transfer stake + reward to disputor
            uint256 totalAmount = res.stakedAmount + res.rewardAmount;
            res.rewardAmount = 0;
            res.stakedAmount = 0;
            IERC20(res.stakeToken).safeTransfer(res.disputor, totalAmount);
            emit StakeClaimed(questionId, res.disputor, totalAmount);
```

This design is flawed, as a timeout does not necessarily mean that the dispute was valid. The `resolve` function has logic to handle such a case when the re-vote process is completed after the re-vote deadline passes:

```
    function resolve(
        bytes32 questionId,
        uint256[] calldata payouts
    ) external onlyRole(RESOLVER_ROLE) nonReentrant whenNotPaused {
    ...
        if (res.receivedAt == 0) {
    ...
        } else if (res.status == DisputeStatus.Disputed) {
    ...
        } else if (res.status == DisputeStatus.TooEarly) {
            // CASE 3: Agent committee's final resolution after a dispute timeout.
            // The dispute window is intentionally skipped here: the resolution was
            // already disputed once (hence TooEarly), the committee is providing
```

```
            // its authoritative answer, and no second dispute window is warranted.
            res.finalPayouts[0] = payouts[0];
            res.finalPayouts[1] = payouts[1];
            res.status = DisputeStatus.Finalized;

            // Already removed from active list during timeout, no need to remove again
            uint256[] memory finalPayouts = new uint256[](2);
            finalPayouts[0] = payouts[0];
            finalPayouts[1] = payouts[1];
            IOracleProxy(oracleProxy).resolve(questionId, finalPayouts);

            emit ResolutionFinalized(questionId, res.finalPayouts);
...
```

It is therefore possible for a dispute to be verified as invalid after the admin sets the resolution status to `TooEarly`. The result is that the disputor will receive their rewards as if their dispute were valid, even though the committee eventually deemed it as invalid. Thus, this design incentivizes bad actors to find ways to delay the re-vote process, since they can directly benefit from timeouts.

Additionally, it is possible for this issue to manifest when the `DisputeResolver` is paused while a resolution is disputed and remains paused until the `reVoteDeadline` has passed. Regardless of the re-vote outcome, the dispute will eventually be considered successful.

A concrete exploitation path exists when a disputor is also a committee member (or colluding with one): instead of proving the original resolution wrong, the attacker only needs to prevent the re-vote from reaching an executable `Approved` state in `ResolutionMultiSig`. If votes remain split or below the approval threshold, the proposal cannot be executed, `reVoteDeadline` passes, and `timeoutDispute()` unconditionally awards the dispute — regardless of whether the committee's actual assessment supported the original resolution.

Recommendation

Consider updating the `timeoutDispute` logic to represent timeouts as stalemates. In this case, the dispute would neither be confirmed nor denied and therefore the outcome should be neutral, with the disputor being able to receive their stake back and the admin being able to recover the rewards. Prevent active committee members from filing disputes or economically benefiting from them (e.g., by excluding committee-role holders from the disputor role), removing the financial incentive to sabotage the re-vote process.

Acknowledgement Comment

This is an intentional design decision to incentivize the committee to make timely decisions on disputes, which benefits the community. Dispute timeouts should carry consequences for inaction, and rewarding the disputor in this case aligns with the protocol's goal of accountability.

We will publish user-facing documentation that clearly describes this behavior so participants understand the timeout dynamics before engaging with the dispute process.

[L-07] Blocklisting token permanently locks claim funds

Description

If the `stakeToken` supports blocklisting (like USDC/USDT on BSC), a blocklisted address causes `safeTransfer` to revert, permanently locking funds in the contract.

`DisputeResolver.sol` — `claim()` (lines 338–377)

```
// Line 369 (dispute succeeded — transfer to disputor):
IERC20(res.stakeToken).safeTransfer(res.disputor, totalAmount);

// Line 375 (dispute failed — transfer to treasury):
IERC20(res.stakeToken).safeTransfer(treasury, stakedAmt);
```

Three blocklisting scenarios cause permanent fund locking:

1. **Disputor blocklisted** after filing dispute — cannot receive claim (line 369)
2. **Treasury blocklisted** — forfeited stakes cannot be sent (line 375)
3. **Contract blocklisted** — ALL outgoing transfers fail

There is no admin rescue mechanism, no alternative recipient, and no way to change the `stakeToken` after the deposit.

Proof of Concept

```
it("Disputor blocklisted after dispute — funds permanently locked", async function () {
  const { dr, usdc, resMultiSig, rewardMgr, disputor, admin, amount } =
    await deployWithBlocklistToken();
  const questionId = await setupOverturnedDispute(
    dr, usdc, resMultiSig, rewardMgr, disputor, "M04a", amount,
  );

  // Dispute was overturned — disputor should get 2× reward
  expect((await dr.getResolution(questionId)).disputeSucceeded).to.be.true;

  // Blocklist disputor BEFORE they claim
  await usdc.addToBlocklist(disputor.address);

  // claim() reverts — transfer to blocklisted address fails
  await expect(dr.connect(disputor).claim(questionId)).to.be.reverted;

  // No admin rescue mechanism exists
  await expect(dr.connect(admin).claim(questionId))
    .to.be.revertedWithCustomError(dr, "NotDisputor");
});

it("Contract blocklisted — ALL claims permanently fail", async function () {
  const { dr, usdc, resMultiSig, rewardMgr, disputor, amount } =
    await deployWithBlocklistToken();
  const questionId = await setupOverturnedDispute(
    dr, usdc, resMultiSig, rewardMgr, disputor, "M04b", amount,
```

```
  );

  // Blocklist the DisputeResolver contract itself
  await usdc.addToBlocklist(await dr.getAddress());

  // ALL claims fail — contract cannot send tokens
  await expect(dr.connect(disputor).claim(questionId)).to.be.reverted;
});

it("Treasury blocklisted — failed dispute claim permanently reverts", async function () {
  const { dr, usdc, resMultiSig, rewardMgr, disputor, treasury, amount } =
    await deployWithBlocklistToken();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("M04c"));

  // Setup: dispute UPHELD (not overturned) — stake goes to treasury
  await dr.connect(resMultiSig).resolve(questionId, [1n, 0n]);
  await usdc.connect(rewardMgr).approve(await dr.getAddress(), amount);
  await dr.connect(rewardMgr).depositRewardPool(questionId, await usdc.getAddress(), amount);
  await usdc.connect(disputor).approve(await dr.getAddress(), amount);
  await dr.connect(disputor).dispute(questionId, 0);
  await time.increase(DISPUTE_WINDOW + 100);
  // Re-vote confirms original → dispute fails (Upheld)
  await dr.connect(resMultiSig).resolve(questionId, [1n, 0n]);

  expect((await dr.getResolution(questionId)).disputeSucceeded).to.be.false;

  // Treasury blocklisted
  await usdc.addToBlocklist(treasury.address);

  // claim() reverts — cannot send forfeited stake to blocklisted treasury
  await expect(dr.connect(disputor).claim(questionId)).to.be.reverted;
});
```

Test result: PASS (3 tests)

Recommendations

1. Add an `emergencyWithdraw()` function callable by `DEFAULT_ADMIN_ROLE` that can redirect funds to an alternative address when the original recipient is blocklisted:

```
function emergencyWithdraw(
    bytes32 questionId,
    address alternativeRecipient
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    Resolution storage res = resolutions[questionId];
    if (res.status != DisputeStatus.Finalized) revert InvalidStatus(questionId, res.status);
    if (!res.claimed) revert AlreadyClaimed(questionId);
    // ... transfer to alternativeRecipient
}
```

1. Implement a pull-payment pattern where recipients can designate an alternative withdrawal address before claiming.

Alternatively, add a `recipient` parameter to `claim()` so the disputor can direct winnings to a non-blocklisted address without requiring administrator intervention.

Acknowledgement Comment

The stake token will be the Opinion native token ($OPN), which does not implement blocklisting functionality. This eliminates the blocklisting risk described in the finding.

[L-08] Dispute slot front-running via single-disputor model

Description

Only one disputor is permitted per resolution. The first caller to `dispute()` claims the slot; all subsequent calls revert with `InvalidStatus`.

`DisputeResolver.sol` — `dispute()` (lines 295–331)

```
// Lines 305–307:
if (res.status != DisputeStatus.Pending) {
    revert InvalidStatus(questionId, res.status);
}
```

A MEV bot or colluding party can monitor the mempool for `dispute()` transactions, front-run with a higher gas price consuming the single slot, and file a weak or intentionally losing dispute to protect the original resolution.

- Legitimate disputors are permanently blocked from challenging resolutions
- Griefing attack: attacker files wrong `DisputeReason` (e.g., `TooEarly` when the real issue is `WrongResult`)
- Attacker loses their stake if the re-vote confirms the original, but prevents a legitimate challenge that might have overturned the result

Proof of Concept

```
it("MEV bot front-runs legitimate disputor, consuming the single slot", async function () {
  const { dr, token, resMultiSig, rewardMgr, disputor, extra: mevBot } = await deployStack();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("M03"));

  await setupPendingWithReward(dr, token, resMultiSig, rewardMgr, "M03", [1n, 0n]);
  await token.mint(mevBot.address, REWARD);

  // MEV bot front-runs (executes first in same block)
  await token.connect(mevBot).approve(await dr.getAddress(), REWARD);
  await dr.connect(mevBot).dispute(questionId, 0);
  expect((await dr.getResolution(questionId)).disputor).to.equal(mevBot.address);

  // Legitimate disputor permanently blocked
  await token.connect(disputor).approve(await dr.getAddress(), REWARD);
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "InvalidStatus");
});

it("Griefing: bot disputes with wrong reason to block real disputes", async function () {
```

```
  const { dr, token, resMultiSig, rewardMgr, disputor, extra: mevBot } = await deployStack();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("M03b"));

  await setupPendingWithReward(dr, token, resMultiSig, rewardMgr, "M03b", [1n, 0n]);
  await token.mint(mevBot.address, REWARD);

  // Bot files TooEarly dispute (reason 1) when the real issue is WrongResult (reason 0)
  await token.connect(mevBot).approve(await dr.getAddress(), REWARD);
  await dr.connect(mevBot).dispute(questionId, 1); // TooEarly reason

  // Legitimate WrongResult disputor blocked
  await token.connect(disputor).approve(await dr.getAddress(), REWARD);
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "InvalidStatus");
});
```

Test result: PASS (2 tests)

Recommendations

Two potential fixes:

1. **Commit-reveal scheme** — Two-phase dispute where disputors commit a hash first, then reveal. This prevents front-running since the MEV bot cannot see the dispute details.
2. **Multiple disputors with highest stake win** — Allow multiple disputes in a bonding curve; highest stake gets priority. This makes front-running economically impractical.

BSC's private mempool submission can serve as an operational mitigation but is not a protocol-level fix.

Acknowledgement Comment

This is a deliberate design decision to prevent abuse. The staking requirement discourages bad actors from frivolously consuming dispute slots. Disputes should be relatively rare and will always be carefully considered and adjudicated by the committee. The economic cost of front-running (losing the stake if the dispute fails) serves as a natural deterrent.

[L-09] OracleProxy preregistration not enforced at initialize

Description

Neither `initialize()` nor `setOracleProxy()` verifies that the provided `OracleProxy` address has the expected interface, accepts the `DisputeResolver` as a caller, or has registered the correct question market.

`DisputeResolver.sol` — `initialize()` (line 209), `setOracleProxy()` (line 577)

```
// Lines 577–582:
function setOracleProxy(address _oracleProxy) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_oracleProxy == address(0)) revert ZeroAddress();
    address oldProxy = oracleProxy;
    oracleProxy = _oracleProxy;
    emit OracleProxyUpdated(oldProxy, _oracleProxy);
}
```

No `IERC165.supportsInterface()` check, no test call, no validation beyond `!= address(0)`.

If the admin sets a wrong oracle address, **all future finalizations fail permanently**. Questions already in the dispute pipeline become unfinalizable. It requires an admin-initiated `setOracleProxy()` to recover, during which more questions pile up in a broken state.

Proof of Concept

```
it("finalize() fails when OracleProxy rejects DisputeResolver as caller", async function () {
  const { dr, oracle, resMultiSig, operator } = await deployStack();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("H03"));

  // No check that OracleProxy accepts DisputeResolver
  expect(await dr.oracleProxy()).to.equal(await oracle.getAddress());

  // Create resolution
  await dr.connect(resMultiSig).resolve(questionId, [1n, 0n]);
  const res = await dr.getResolution(questionId);
  await time.increaseTo(res.disputeDeadline);

  // Simulate OracleProxy rejecting unregistered caller
  await oracle.setShouldRevert(true);
  await expect(
    dr.connect(operator).finalize(questionId),
  ).to.be.revertedWith("MockOracleProxy: forced revert");
});
```

Test result: PASS

Recommendations

1. Add an interface check in `setOracleProxy()` and `initialize()`. At minimum, perform a `staticcall` to verify the oracle supports `IOracleProxy.resolve.selector`:

```
function setOracleProxy(address _oracleProxy) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_oracleProxy == address(0)) revert ZeroAddress();
    // Verify interface
    try IERC165(_oracleProxy).supportsInterface(type(IOracleProxy).interfaceId) returns
(bool supported) {
        if (!supported) revert InvalidOracleProxy();
    } catch {
        revert InvalidOracleProxy();
    }
    address oldProxy = oracleProxy;
```

```
    oracleProxy = _oracleProxy;
    emit OracleProxyUpdated(oldProxy, _oracleProxy);
}
```

1. Consider requiring a two-step oracle update with a test finalization on a known question to confirm round-trip communication works before committing the new address.

Acknowledgement Comment

We acknowledge this finding but will skip the implementation for now. Oracle proxy validation will be considered in a future iteration.

[L-10] Reward_manager non-deposit creates permanent dispute censorship

Description

The `dispute()` function requires `res.rewardAmount > 0` (line 315). The reward is set exclusively by `REWARD_MANAGER_ROLE` via `depositRewardPool()` (line 443). If the reward manager never deposits — whether by omission, compromise, or operational failure — the dispute function is permanently blocked for that question.

`DisputeResolver.sol` — `dispute()` (line 315), `depositRewardPool()` (line 443)

```
// Line 315 in dispute():
if (res.rewardAmount == 0) {
    revert NoRewardDeposited(questionId);
}
```

The 24-hour dispute window runs from the moment `resolve()` is called (line 246), not from when the reward is deposited. A delayed deposit effectively shortens the window; a missing deposit eliminates it entirely.

Additionally, `depositRewardPool()` does not enforce a minimum remaining time after deposit. A deposit accepted at `disputeDeadline - 1` passes the current check but leaves zero practical time for disputors to act, meaning even a technically on-time deposit can effectively eliminate the dispute window.

- A compromised `REWARD_MANAGER` can censor all disputes by simply not depositing.
- Legitimate resolutions become unchallengeable.
- Combined with a compromised `RESOLVER`, any resolution can be forced through.

Proof of Concept

```
it("dispute() permanently blocked when REWARD_MANAGER never deposits", async function () {
  const { dr, token, resMultiSig, operator, disputor } = await deployStack();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("H02"));
```

```
  // Resolution created — but NO reward deposit
  await dr.connect(resMultiSig).resolve(questionId, [1n, 0n]);
  const res = await dr.getResolution(questionId);

  // Disputor has funds and approval
  await token.connect(disputor).approve(await dr.getAddress(), REWARD);

  // dispute() reverts — no reward deposited
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "NoRewardDeposited");

  // Wait half the window — still blocked
  await time.increaseTo(res.receivedAt + BigInt(DISPUTE_WINDOW / 2));
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "NoRewardDeposited");

  // Window expires — now DisputeWindowClosed
  await time.increaseTo(res.disputeDeadline + 1n);
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "DisputeWindowClosed");

  // Operator finalizes unchallenged
  await dr.connect(operator).finalize(questionId);
  expect((await dr.getResolution(questionId)).status).to.equal(5n);
});
```

Test result: PASS

Recommendations

Three potential fixes:

1. **Require deposit atomically with** `resolve()` — Have the `ResolutionMultiSig` supply the reward in the same transaction as the resolution submission, guaranteeing the reward exists before the window starts.
2. **Allow disputes without reward deposit** — If no reward is deposited, let disputors stake a default amount. Handle reward accounting at claim time.
3. **Extend deadline until deposit** — Do not start the dispute window until `depositRewardPool()` is called, ensuring the full 24 hours are available post-deposit:

```
function depositRewardPool(...) external {
    // ...existing logic...
    res.receivedAt = block.timestamp;
    res.disputeDeadline = block.timestamp + disputeWindow;
}
```

[L-11] OracleProxy.resolve() revert permanently blocks finalization

Description

Both finalization paths call `IOracleProxy(oracleProxy).resolve()` as an external call without any fallback:

`DisputeResolver.sol` — `_finalize()` (line 821), `resolve()` Too Early case (line 279)

```
// Line 821 in _finalize():
IOracleProxy(oracleProxy).resolve(questionId, payouts);

// Line 279 in resolve() TooEarly case:
IOracleProxy(oracleProxy).resolve(questionId, finalPayouts);
```

If `OracleProxy.resolve()` reverts for any reason (bug, upgrade, access control rejection, gas limit), the resolution becomes permanently stuck. There is no fallback, retry mechanism, or admin override to bypass a broken oracle.

- All pending/disputed resolutions become unfinalizable.
- Disputor stakes and reward pools are permanently locked.
- The protocol is fully halted until OracleProxy is fixed externally.

Proof of Concept

```
it("finalize() reverts permanently when OracleProxy.resolve() reverts", async function () {
  const { dr, oracle, resMultiSig, operator } = await deployStack();
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("H01"));
  await dr.connect(resMultiSig).resolve(questionId, [1n, 0n]);
  const res = await dr.getResolution(questionId);
  await time.increaseTo(res.disputeDeadline);

  // Oracle starts reverting
  await oracle.setShouldRevert(true);

  // finalize() permanently blocked
  await expect(
    dr.connect(operator).finalize(questionId),
  ).to.be.revertedWith("MockOracleProxy: forced revert");

  // Resolution stuck in Pending
  expect((await dr.getResolution(questionId)).status).to.equal(0n);

  // Only recovers after oracle is fixed externally
  await oracle.setShouldRevert(false);
  await dr.connect(operator).finalize(questionId);
  expect((await dr.getResolution(questionId)).status).to.equal(5n);
});

it("TooEarly re-resolution also blocked by reverting oracle", async function () {
  const { dr, token, oracle, resMultiSig, operator, rewardMgr, disputor } = await
deployStack();
```

```
  const questionId = ethers.keccak256(ethers.toUtf8Bytes("H01b"));
  const { disputeDeadline } = await setupPendingWithReward(
    dr, token, resMultiSig, rewardMgr, "H01b", [1n, 0n],
  );

  // Dispute → timeout → TooEarly
  await token.connect(disputor).approve(await dr.getAddress(), REWARD);
  await dr.connect(disputor).dispute(questionId, 0);
  const resAfterDispute = await dr.getResolution(questionId);
  await time.increaseTo(resAfterDispute.reVoteDeadline);
  await dr.connect(operator).timeoutDispute(questionId);
  expect((await dr.getResolution(questionId)).status).to.equal(4n); // TooEarly

  // Oracle starts reverting
  await oracle.setShouldRevert(true);

  // TooEarly re-resolution blocked because resolve() calls IOracleProxy at line 279
  await expect(
    dr.connect(resMultiSig).resolve(questionId, [0n, 1n]),
  ).to.be.revertedWith("MockOracleProxy: forced revert");
});
```

Test result: PASS (2 tests)

Recommendations

Two potential fixes:

1. Wrap the `IOracleProxy.resolve()` call in a try/catch. On failure, emit a `FinalizationFailed` event and allow the admin to retry or update the oracle address:

```
try IOracleProxy(oracleProxy).resolve(questionId, payouts) {
    // success
} catch {
    emit FinalizationFailed(questionId, payouts);
    // Allow admin retry via emergencyFinalize()
}
```

1. Add an `emergencyFinalize()` function callable by `DEFAULT_ADMIN_ROLE` that marks the resolution as Finalized internally without calling the oracle, allowing fund recovery when the oracle is broken.

Acknowledgement Comment

The purpose of the contract is to call resolve on the OracleProxy. If resolve cannot be called, it is better to revert the entire transaction than to manage state manually in a try/catch block, which could lead to inconsistencies between DisputeResolver and OracleProxy state.

[L-12] Operator pause-finalize combo eliminates dispute window

Severity

Impact: Medium

Likelihood: Medium

Description

An `OPERATOR_ROLE` holder can completely eliminate the dispute window by pausing the contract during the 24-hour dispute period and unpausing after the deadline expires.

The `dispute()` function has the `whenNotPaused` modifier (line 298), so disputors are blocked while paused. After unpausing once the deadline has passed, the operator can immediately call `finalize()` (line 388).

`DisputeResolver.sol` — `pause()` (line 587), `unpause()` (line 594), `finalize()` (line 388), `dispute()` (line 295)

A second exploitation path targets the re-vote window (Scenario B): if an attacker front-runs an incoming `pause()` call with a `dispute()` transaction, `reVoteDeadline` is set relative to `block.timestamp` at that moment. During the pause, `resolve()` for Case 2 (re-vote submission) is also gated by `whenNotPaused`, so the AI committee cannot record their re-vote result on-chain. Once `reVoteDeadline` expires while the contract remains paused, `timeoutDispute()` must be called, which unconditionally sets `disputeSucceeded = true` and allows the attacker to claim `stakedAmount + rewardAmount` — a near-risk-free reward theft without the committee ever adjudicating the dispute. If the operator avoids calling `timeoutDispute()` to prevent unfairly awarding the disputor (Scenario B consequence), the resolution remains permanently stuck in `Disputed` status and its final payout is never forwarded to `OracleProxy`, blocking market settlement indefinitely.

Attack Flow:

1. `ResolutionMultiSig` submits resolution → 24-hour dispute window starts
2. 6 hours in, `OPERATOR` calls `pause()`
3. Disputor calls `dispute()` → **reverts** (`Pausable: paused`)
4. 18+ hours pass while the contract is paused
5. `OPERATOR` calls `unpause()` after `disputeDeadline` has passed
6. `OPERATOR` calls `finalize()` immediately
7. Resolution finalized — **zero effective dispute window**

Complete centralization risk. A compromised or colluding operator can force any resolution through without community oversight. Combined with a compromised `RESOLVER`, this enables arbitrary oracle manipulation in the prediction market.

Proof of Concept

```
it("OPERATOR skips dispute window via pause → wait → unpause → finalize", async function () {
  const { dr, token, resMultiSig, operator, rewardMgr, disputor } = await deployStack();
  const { questionId, disputeDeadline } = await setupPendingWithReward(
    dr, token, resMultiSig, rewardMgr, "C01", [1n, 0n],
  );

  // 1. Operator pauses 6h into the 24h window
  const res = await dr.getResolution(questionId);
  await time.increaseTo(res.receivedAt + BigInt(6 * 3600));
  await dr.connect(operator).pause();

  // 2. Disputor cannot dispute while paused
  await token.connect(disputor).approve(await dr.getAddress(), REWARD);
  await expect(dr.connect(disputor).dispute(questionId, 0)).to.be.reverted;

  // 3. Time passes beyond deadline while paused
  await time.increaseTo(disputeDeadline + 1n);

  // 4. Operator unpauses and finalizes immediately
  await dr.connect(operator).unpause();

  // 5. Disputor too late — dispute window expired
  await expect(
    dr.connect(disputor).dispute(questionId, 0),
  ).to.be.revertedWithCustomError(dr, "DisputeWindowClosed");

  // 6. Finalization succeeds — full dispute window was eliminated
  await dr.connect(operator).finalize(questionId);
  expect((await dr.getResolution(questionId)).status).to.equal(5n); // Finalized
});
```

Test result: PASS

Recommendations

Three potential fixes:

1. **Pause-aware deadline extension** — Track cumulative paused duration and add it to `disputeDeadline` on unpause, ensuring the full window is available:

```
uint256 private _pausedAt;
uint256 private _totalPausedDuration;

function pause() external onlyRole(OPERATOR_ROLE) {
    _pausedAt = block.timestamp;
    _pause();
}

function unpause() external onlyRole(OPERATOR_ROLE) {
    _totalPausedDuration += block.timestamp - _pausedAt;
```

```
    _unpause();
    // Extend all active dispute deadlines by paused duration
}
```

1. **Remove `whenNotPaused` from `dispute()`** — Let disputors file disputes even while paused, matching how `withdrawRewardPool()` (line 487) intentionally omits `whenNotPaused`.
2. **Timelock pause/unpause** — Require a timelock delay on `unpause()` that exceeds the remaining dispute window, preventing instant finalization after unpause.
3. Remove `whenNotPaused` from `resolve()` for the re-vote case (Case 2) so the AI committee can still submit re-vote results during a pause, preventing the timeout path from being weaponized against the re-vote window.

Acknowledgement Comment

We mitigate this through operational controls:

The operator role uses a KMS signer, reducing the risk of key compromise. The admin role (multi-sig wallet) can revoke the operator role in the event of systematic failure of the KMS system on AWS. The admin role itself is governed by a multi-sig wallet, preventing unilateral action.